

Machine Learning in Finance

Week-1

Start coding or [generate](#) with AI.

NumPy

The main python library used in numerical data analysis is NumPy (or Numpy). It is a Linear Algebra Library for Python. It is a very important library for Data Science with Python mainly because Numpy is incredibly fast, as it has bindings to C libraries.

It provides a high-performance multidimensional array object, and tools for working with these arrays.

This practical gives only an introduction to Numpy.

Installation NumPy

In the case of using the Anaconda distribution, the Numpay has already been installed. If you have any problem with Numpay functionality, it is highly recommended to re-install Numpay using the following command:

```
install numpy
```

```
# Import NumPy
import numpy as np
```

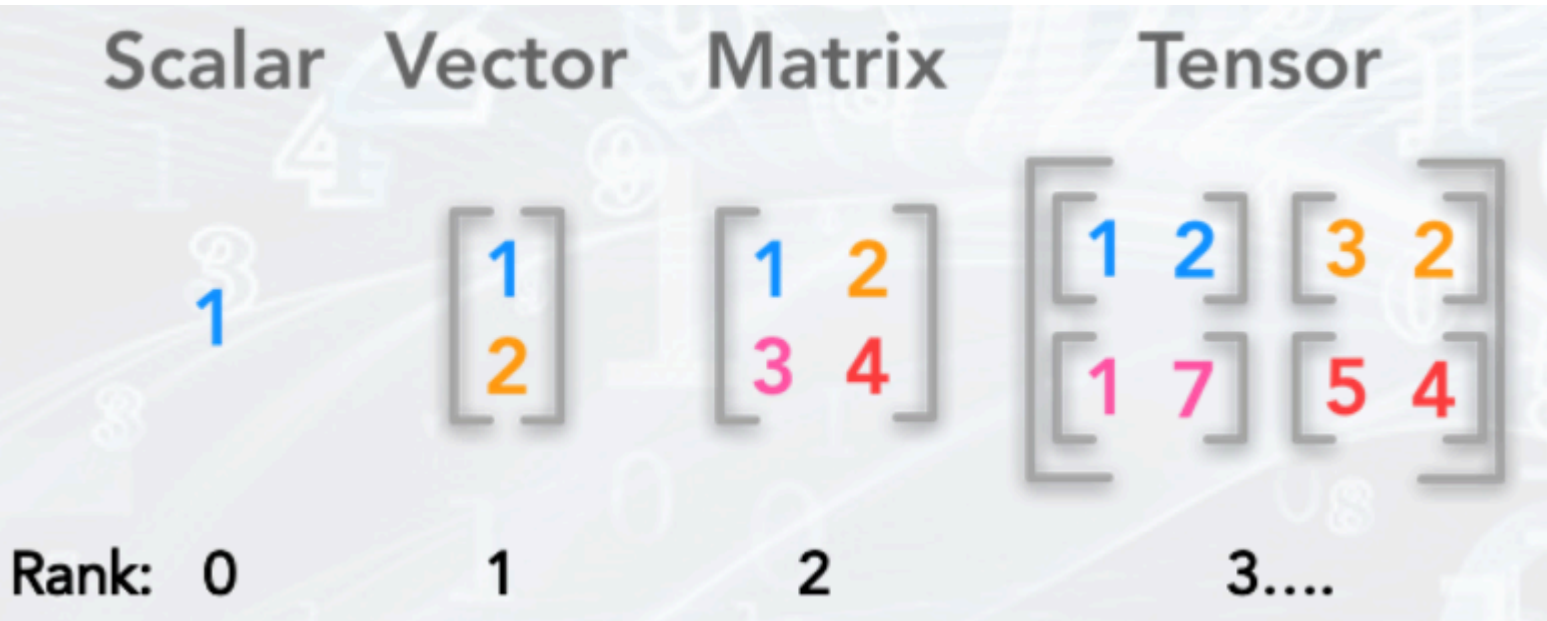
```
import numpy as np
```

– From this point on, a cell marked simply with "#" is shown to allow you to compare your result with the

One of the most popular features of the Numpy library is the capability of operations with arrays of different sizes (or ranks).

Dimensions of arrays:

- vectors - One dimensional arrays,
- matrices - Two dimensional arrays,
- tensor - Three or more arrays.



49.png

```
# Create a rank 1 array
a = np.array([1, 2, 3])
```

```
a = np.array([1,2,3])
```

```
#
```

```
# Print the type and shape the array

print(type(a))
print(a.shape)
```

```
print(type(a))
print(a.shape)
```

```
<class 'numpy.ndarray'>
(3,)
```

```
#
```

```
# Print the values
# The numbering of Numpy array elements starts with -0-, just like in C.

print(a[0], a[1], a[2])

# or

print(a)
```

```
print(a[0], a[1], a[2])
print(a)
```

```
1 2 3
[1 2 3]
```

```
#|
```

```
# Change an element of the array

a[0] = 5

print(a)
```

```
a[0] = 5
print(a)
```

```
[5 2 3]
```

```
#|
```

```
# Create a rank 2 array

b = np.array([[1,2,3],[4,5,6]])
b
```

```
b = np.array([[1,2,3],[4,5,6]])
b
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
#
```

```
# Print an element of the array
# Remember: the numbering of Numpy array elements starts with -0-, just like in C.

print(b[1, 0])
```

```
print(b[1,0])
```

```
4
```

```
#
```

▼ **arange**

The arange() method is used to return an evenly spaced values within a given interval. The method has the following parameters:

- **start** parameter (inclusive in the sequence)
- **end** parameter (exclusive in the sequence)
- **step size** (optional parameter used to find the next element in the sequence)

```
# Note the end item is excluded.

np.arange(10,20)
```

```
np.arange(10,20)
```

```
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

```
#
```

```
# Using negative step factor gives a decreasing sequence.

np.arange(30,1,-3)
```

```
np.arange(30,1,-3)
```

```
array([30, 27, 24, 21, 18, 15, 12,  9,  6,  3])
```

```
#
```

Let us compare the performance of Numpay and Python arrays.

```
# timeit - executes the code several times and gives statistics.
```

▼ Now just press 'CTRL' and 'ENTER' keys on the next 3 cells.

```
%timeit np.arange(0, 1000)
```

```
1.7 µs ± 479 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

```
%timeit range(0, 1000)
```

```
96 ns ± 23.5 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
```

```
# The command above does not create an array of a thousand numbers
```

```
%timeit list(range(1000))
```

```
24.7 µs ± 6.54 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
```

```
# Let's measure how much faster NumPy performs arithmetic operations
```

Now just press 'CTRL' and 'ENTER' keys on the next 3 cells.

```
numbers = list(range(1000000))
array = np.arange(1000000, dtype=np.int64)
```

```
%timeit [x * x for x in numbers]

69.3 ms ± 1.11 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
%timeit array * array

958 µs ± 125 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

Write your conclusion in the next cell :

#NumPy arrays demonstrate significantly faster performance for arithmetic operations.

Numpy also provides many functions to create arrays

```
# Create a 2-d array array of all zeros

a = np.zeros((2,2))
a
```

```
a = np.zeros((2,2))
a

array([[0., 0.],
       [0., 0.]])
```

#

```
# Create a 2-d array array of all ones

b = np.ones((3,4))
b
```

```
b = np.ones((3,4))
b

array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

#

```
# Create a 4x4 identity matrix

# Return a 2-D array with ones on the diagonal and zeros elsewhere.
# Takes only a single parameter which determines dimension of the square matrix.

np.eye(4)
```

```
np.eye(4)

array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
```

#

```
# Create an array filled with random values

e = np.random.random((2,2))
e
```

```
e = np.random.random((2,2))
e

array([[0.66377872, 0.23965806],
       [0.02884335, 0.56486329]])
```

#

```
# Generate eleven elements between 0 and 20

np.linspace(0,20,11)
```

```
np.linspace(0,20,11)

array([ 0.,  2.,  4.,  6.,  8., 10., 12., 14., 16., 18., 20.] )
```

#

```
# Method can be used to generate finer granular sequence

np.linspace(0,20,30)
```

```
np.linspace(0,20,30)

array([ 0.          ,  0.68965517,  1.37931034,  2.06896552,  2.75862069,
        3.44827586,  4.13793103,  4.82758621,  5.51724138,  6.20689655,
        6.89655172,  7.5862069 ,  8.27586207,  8.96551724,  9.65517241,
       10.34482759, 11.03448276, 11.72413793, 12.4137931 , 13.10344828,
       13.79310345, 14.48275862, 15.17241379, 15.86206897, 16.55172414,
       17.24137931, 17.93103448, 18.62068966, 19.31034483, 20.          ])
```

#

Array Indexing

Slicing: Similar to Python lists, numpy arrays can be sliced. Since arrays may be multidimensional, you must specify a slice for each dimension of the array:

```
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

a
```

```
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
a

array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

#

```
# Use slicing to pull out the subarray consisting of the first 2 rows and columns 1 and 2

b = a[:2, 1:3]

b
```

```
b = a[:2, 1:3]
b
```

```
array([[2, 3],
       [6, 7]])
```

```
#
```

```
# A slice of an array is a view into the same data, so modifying it
# will modify the original array.

print(a[0, 1])
```

```
print(a[0,1])
```

```
2
```

```
#
```

```
# b[0, 0] is the same of a[0, 1]
# change b[0, 0] and print a[0, 1]

b[0, 0] = 88
print(a)
```

```
b[0, 0] = 88
print(a)
```

```
[[ 1 88  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

```
#
```

▼ Datatypes

Every NumPy array is a grid of elements of the **same type**. Numpy provides a large set of numeric datatypes that you can use to construct arrays. Numpy tries to guess a datatype when you create an array, but you can explicitly specify the type using an optional argument.

```
x = np.array([1, 2])
print(x.dtype)
```

```
x = np.array([1,2])
print(x.dtype)
```

```
int64
```

```
#
```

```
x = np.array([1.0, 2.0])
print(x.dtype)
```

```
x = np.array([1.0,2.0])
print(x.dtype)
```

```
float64
```

```
#
```

```
x = np.array([1, 2], dtype=np.int16)
print(x.dtype)
```

```
x = np.array([1,2], dtype=np.int16)
print(x.dtype)
```

int16

#

Array Math

```
# Basic mathematical functions operate elementwise on arrays
```

Now just press 'CTRL' and 'ENTER' keys on the next cell.

```
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)
print(x, '\n')      # '\n' - Line break symbol
print(y)
```

[[1. 2.]
 [3. 4.]]

[[5. 6.]
 [7. 8.]]

```
# Compare the result. Is it the same?

print(x + y)
print('\n')
print(np.add(x, y))
```

```
print(x + y)
print('\n')
print(np.add(x, y))
```

[[6. 8.]
 [10. 12.]]

[[6. 8.]
 [10. 12.]]

#

```
print(x - y, '\n')
print(np.subtract(x, y))
```

```
print(x - y, '\n')
print(np.subtract(x,y))
```

[[-4. -4.]
 [-4. -4.]]

[[-4. -4.]
 [-4. -4.]]

#

```
# Elementwise product; both produce the array

print(x * y, '\n')
print(np.multiply(x, y))
```

```
print(x * y, "\n")
print(np.multiply(x, y))
```

[[5. 12.]
 [21. 32.]]

```
[[ 5. 12.]
 [21. 32.]]
```

```
#
```

```
# Elementwise division; both produce the array

print(x / y)
print('\n')
print(np.divide(x, y))
```

```
print(x / y)
print('\n')
print(np.divide(x, y))

[[0.2      0.33333333]
 [0.42857143 0.5      ]]

[[0.2      0.33333333]
 [0.42857143 0.5      ]]
```

```
#
```

```
# Elementwise square root; produces the array

print(np.sqrt(x))
```

```
print(np.sqrt(x))

[[1.      1.41421356]
 [1.73205081 2.      ]]
```

```
#
```

```
# Exponentiation

x**3
```

```
#
```

```
# Reciprocal Value

1/x
```

```
1/x

array([[1.      , 0.5      ],
       [0.33333333, 0.25    ]])
```

```
#
```

Matrix Multiplication

```
# Note that the above was elementwise multiplication, not matrix multiplication.
# To receive the matrix multiplication, we use the dot function.
```

Now just press 'CTRL' and 'ENTER' keys on the next cell.

```
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
```

```
w = np.array([11, 12])
print(x, '\n\n', y, '\n\n', v, '\n\n', w)

[[1 2]
 [3 4]]

[[5 6]
 [7 8]]

[ 9 10]

[11 12]
```

Matrix Multiplication – Dot Product (Scalar Product)

$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \cdot \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = x_1y_1 + x_2y_2 + x_3y_3$$

Not Elementwise Multiplication

```
# Inner product of vectors

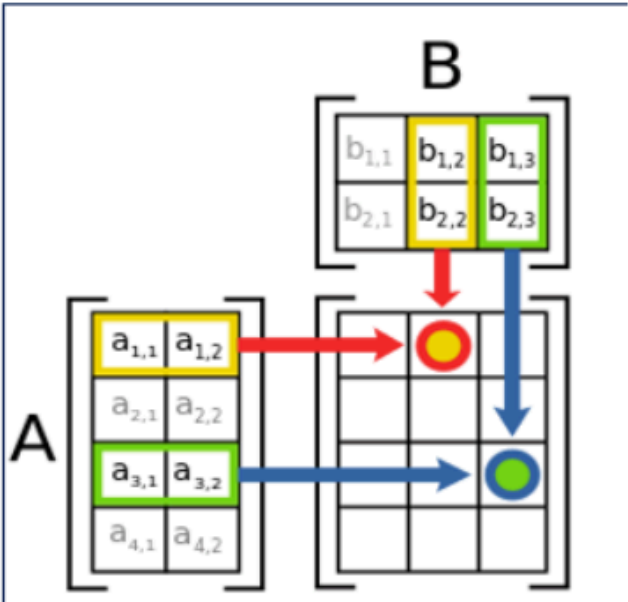
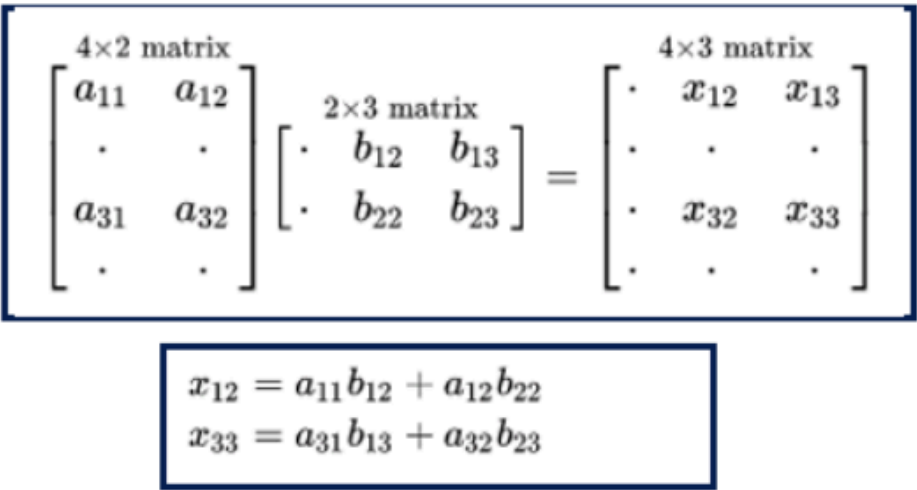
print(v.dot(w))
print(np.dot(v, w))
```

```
print(v.dot(w))
print(np.dot(v,w))
```

219
219

#

Matrix Multiplication



Video about Matrix Multiplication:

Watch this video carefully at **home** and remember how matrices are multiplied

<https://www.youtube.com/watch?v=2spTnAiQg4M>

```
# Matrix / vector product

print(x.dot(v))
print('\n')
print(np.dot(x, v))
```

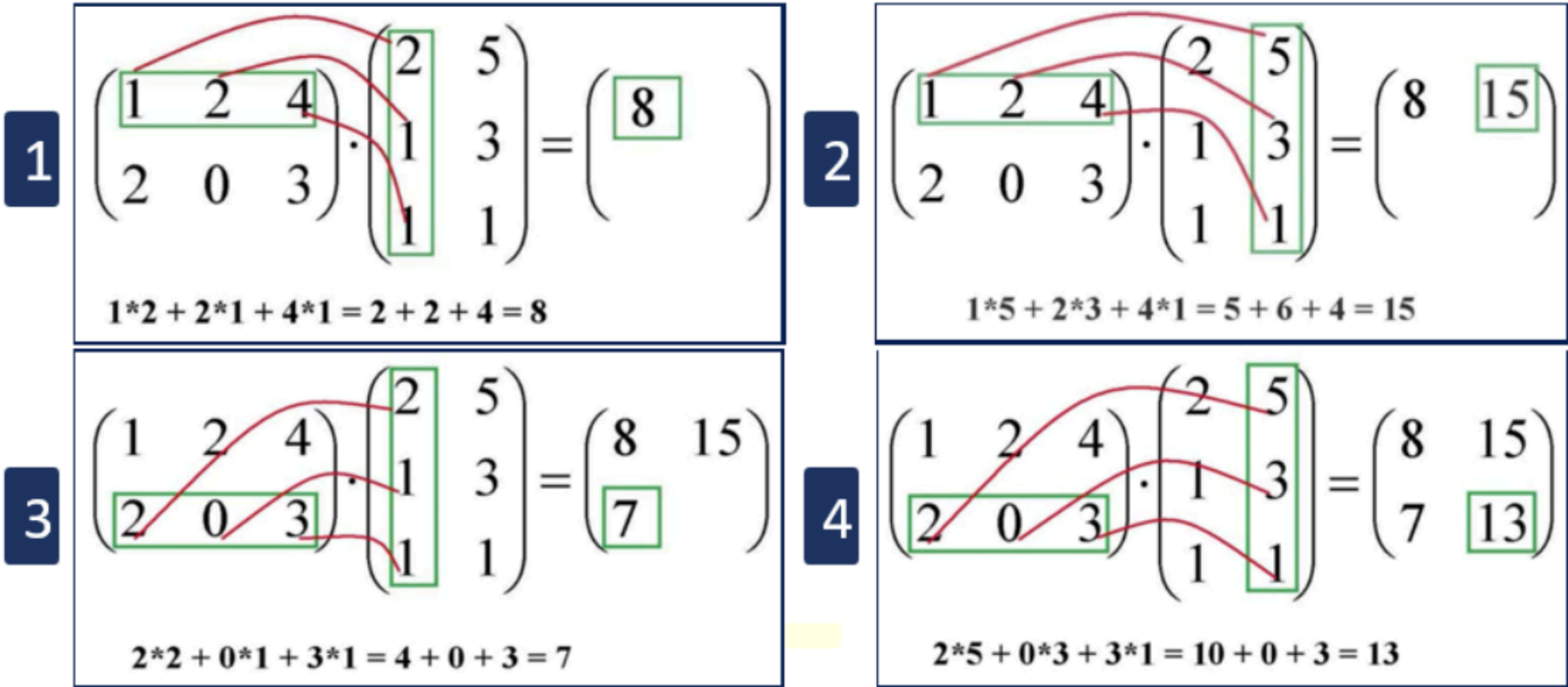
```
print(x.dot(v))
print('\n')
print(np.dot(x,v))
```

[29 67]

[29 67]

#

an example of matrix multiplication:



```
# Matrix / matrix product

print(x.dot(y))
print('\n')
print(np.dot(x, y))
```

```
print(x.dot(y))
print('\n')
print(np.dot(x, y))
```

[[19 22]
 [43 50]]

[[19 22]
 [43 50]]

#

Random number array generator

```
# Generate a random number array from uniform distribution in the interval [0,1].

np.random.rand(4)
```

```
np.random.rand(4)

array([0.97779602, 0.35726373, 0.00731862, 0.96525621])
```

#

```
# Generate a 2-d array of random numbers

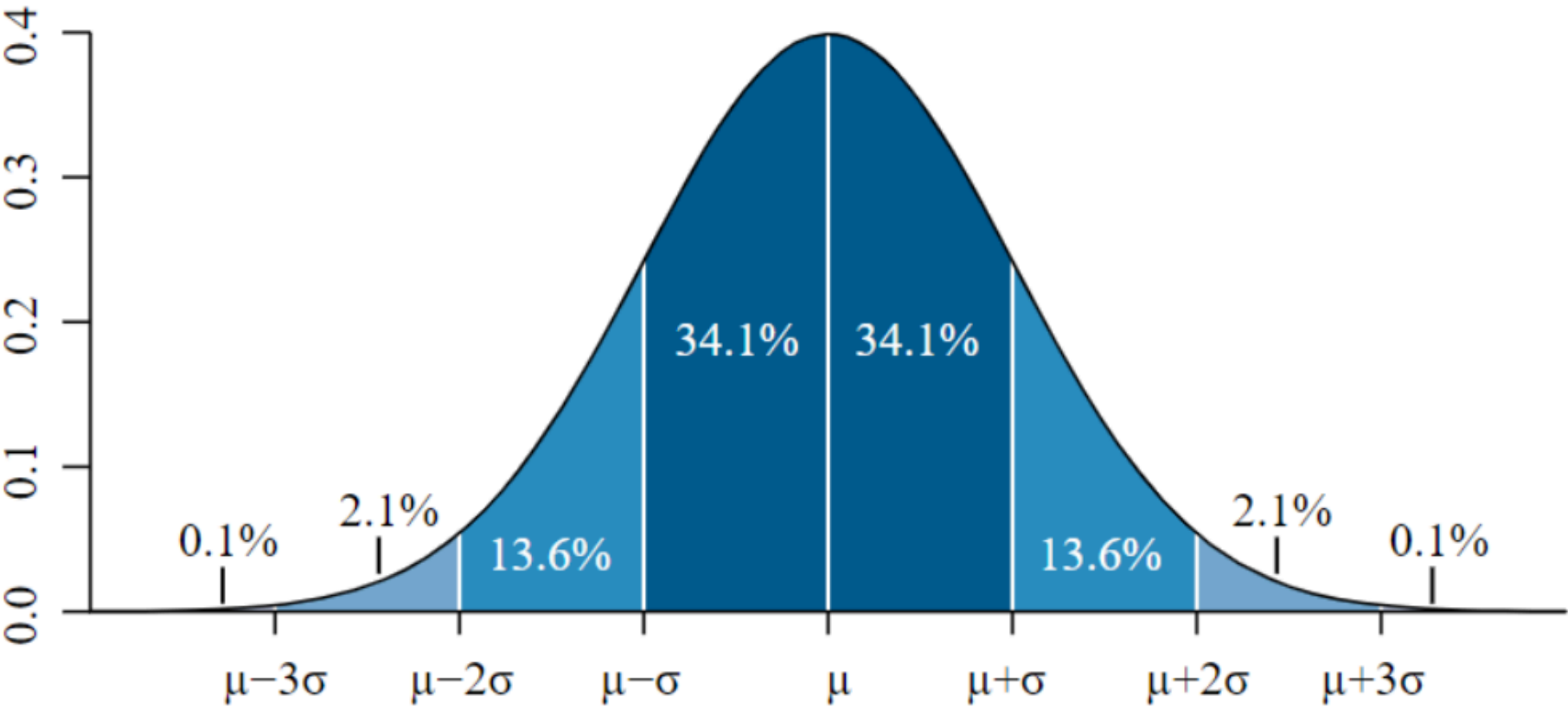
np.random.rand(4,4)
```

```
np.random.rand(4,4)

array([[0.3443822 , 0.86861661, 0.11122923, 0.20384107],
       [0.58017073, 0.22178746, 0.33080242, 0.1540003 ],
       [0.83600201, 0.40548829, 0.53967684, 0.1777579 ],
       [0.95525917, 0.34201506, 0.4757569 , 0.94257653]])
```

#

Normal distribution (Gaussian distribution)



```
# Generates a 2-d random array from a normal distribution.
# Unlike rand, which is uniform, randn() returns a sample (or samples) from the "standard normal" distribution.

np.random.randn(4,4)
```

```
np.random.rand(4,4)

array([[0.13720725, 0.90161125, 0.49524542, 0.9787131 ],
       [0.80876645, 0.57211804, 0.60064538, 0.30640389],
       [0.7409401 , 0.41789317, 0.75331199, 0.16989344],
       [0.54554708, 0.60317044, 0.13363361, 0.91517523]])
```

#

```
# Generate a single random integer between 1 and 00.

np.random.randint(1,300)
```

```
np.random.randint(1,300)
```

114

#

```
# Generate a sequence (15) of random numbers between 1 and 200
np.random.randint(1,200,15)
```

```
np.random.randint(1,200,15)
array([119, 120,  24,  52, 139, 180, 118,  69,   3, 103, 135, 160, 125,
       157, 140])
```

```
#
```

Reshape

The shape (dimensions) of the array can be changed using the reshape() method This returns an array containing the same data with a new shape.

```
p = np.arange(0,15)
p
```

```
p = np.arange(0,15)
p
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

```
#
```

```
# Convert the 15 element 1-d array into a 2-d array.
p = p.reshape(3,5)
p
```

```
p = p.reshape(3,5)
p
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

```
#
```

```
# Both operations above can be combined into one line.
s = np.arange(12).reshape(3, 4)
s
```

```
s = np.arange(12).reshape(3, 4)
s
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
#
```

▼

Transposing a Matrix

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

$$(1.01 \quad 2.02 \quad 3.03 \quad 4.04)^T = \begin{pmatrix} 1.01 \\ 2.02 \\ 3.03 \\ 4.04 \end{pmatrix}$$

$$(x \quad y \quad z)^T \rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

```
# to transpose a matrix, simply use the T attribute of an array object  
print(p.T)
```

```
print(p.T)  
  
[[ 0  5 10]  
 [ 1  6 11]  
 [ 2  7 12]  
 [ 3  8 13]  
 [ 4  9 14]]
```

```
#
```

```
# Create an empty matrix with the same shape as p  
  
k = np.empty_like(p)  
k
```

The command does not create an "empty" matrix in the sense of being filled with zeros or having no values. Instead, it returns a new array with the same shape and data type as p, but without initialising the elements — meaning the array elements contain random values that happened to be in memory at the time of allocation.

```
k = np.empty_like(p)  
k  
  
array([[ 137983965211616,    505743584,          0,  
         0,          0],  
       [7310593858020254331, 3616445622929465956, 6067811242765987126,  
        3688512107902023985, 6501283836426597422],  
       [7309453675965983778, 8315168162784306286, 8367752027310484831,  
        7954801838398993778, 2459029315949324647]])
```

```
#
```

```
# Stack 4 copies of vector(v) on top of each other:  
  
v = np.array([1, 0, 1])  
print(v, '\n')  
  
vv = np.tile(v, (4, 1))  
print(vv)
```

```
v = np.array([1, 0, 1])  
print(v, '\n')
```

```
vv = np.tile(v, (4, 1))
print(vv)
```

```
[1 0 1]

[[1 0 1]
 [1 0 1]
 [1 0 1]
 [1 0 1]]
```

```
#
```

Functions

```
# print again the a matrix which we use above
a
```

```
a
array([[ 1, 88,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

```
#
```

```
# Numpy provides many useful functions for performing computations on arrays; one of the most useful is sum:
```

```
# Compute sum of all elements
print(np.sum(a))
```

```
print(np.sum(a))

164
```

```
#
```

```
# Compute sum of each column
print(np.sum(a, axis=0))
```

```
print(np.sum(a, axis=0))

[ 15 104  21  24]
```

```
#
```

```
# Compute sum of each row
print(np.sum(a, axis=1))
```

```
print(np.sum(a, axis=1))

[96 26 42]
```

```
#
```

```
# Find the max value in the array
a.max()
```

```
a.max()
```

```
np.int64(88)
```

```
#
```

```
# Find the minimum value in the array
a.min()
```

```
a.min()
```

```
np.int64(1)
```

```
#
```

```
# Find the index of the maximum value
a.argmax()
```

```
a.argmax()
```

```
np.int64(1)
```

```
#
```

```
# Find the index of the minimum value
a.argmin()
```

```
a.argmin()
```

```
np.int64(0)
```

```
#
```

Shape

The size of a Numpy array can be retrieved using the attribute named "shape". Please note it is not a method. Attribute does not use "()" when evoked.

```
# Size of a vector
arr = np.arange(10)
print(arr)
print(arr.shape)
```

```
arr = np.arange(10)
print(arr)
print(arr.shape)
```

```
[0 1 2 3 4 5 6 7 8 9]
(10,)
```

```
#
```

```
# print again the a matrix which we used above
print(a)
print(a.shape)
```

```
print(a)
print(a.shape)
```

```
[[ 1 88  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
(3, 4)
```

```
#
```

The shape of an array can be changed with various commands. Note that the following three commands all return a modified array, but do not change the original array:

```
# returns the array, flattened
a.ravel()
```

```
a.ravel()
```

```
array([ 1, 88,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

```
a.ravel()
```

```
array([ 1, 88,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

```
# returns the array with a modified shape
a.reshape(6, 2)
```

```
a.reshape(6,2)
```

```
array([[ 1, 88],
       [ 3,  4],
       [ 5,  6],
       [ 7,  8],
       [ 9, 10],
       [11, 12]])
```

```
#
```

```
print(a.shape)
print(a.T.shape)
```

```
print(a.shape)
print(a.T.shape)
```

```
(3, 4)
(4, 3)
```

```
#
```

Change matrix a to **2-d** array with 1 column. Notice the two sets of brackets for a 2-d array with one column.

Save it in another array.

Check the shape attribute value.

```
a3 = a.reshape(12,1)
print(a3)
print('\n')
print(a3.shape)
```

```
a3 = a.reshape(12,1)
print(a3)
print('\n')
print(a3.shape)
```

```
[[ 1]
 [88]
 [ 3]
 [ 4]
```

25/09/2026, 21:15Week1_NumPy_ML_in_Finance_Proof_Colab.ipynb - Colab

```
[ 5]
[ 6]
[ 7]
[ 8]
[ 9]
[10]
[11]
[12]]

(12, 1)

#
```

(!) From the point of view of mathematics - this is a 1-d vector. However, in terms of Numpy, it is a 2-dimensional array.

```
#SID is 2520731, so the last two digits are 31.

#1)Create a vector using np.arange.
a = np.arange(31)
print(a)

#2)Change a to a 2-D array with 1 row
a1 = a.reshape(1, 31)
print(a1)

#3)Save it in another array. Print the array.
a2 = a1
print(a2)

#4)Check the shape attribute value.
print(a2.shape)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30]
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30]]
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30]]
(1, 31)
```

✓ Lab Logbook Requirement:

1) Create a vector using np.arange.

Determine the number of the vector elements using the following method: Take the last two digits from your SID. It should be from 00 to 99. If this number is 10 or more, it becomes the required number of the vector elements. If it is less than 10, add 100 to your number.

For example, if your SID is 2287467, and the last two digits are 67, which is greater than 10. The required number is 67. If your SID is 2287407, and the last two digits are 07, which is less than 10. The required number is 107.

Then,

- 2. Change matrix a to 2-d array with 1 row. Print the array. You should have the two sets of brackets for a 2-d array with one row.
- 3. Save it in another array. Print the array.
- 4. Check the shape attribute value.
- 5. Add the code and result to your Lab Logbook